

set up the testing environment, and conduct the tests. We will also discuss common issues that may arise during testing and provide troubleshooting tips. By the end of this guide, you will have a solid understanding of how to test your app on foldable screens, helping you create an app that delivers an exceptional user experience across all devices.

Coding changes for foldable screens

To accommodate foldable screens, certain modifications in your app's code may be required. Here are some important changes:



- **Configuration Changes:** Android triggers a configuration change event when the screen size or orientation changes. By default, this event causes the app to restart, interrupting the user's experience. You can handle configuration changes manually in your app's code to maintain state and provide a smooth transition. This can be done by overriding the `onConfigurationChanged()` method in your Activity class and implementing your logic to handle changes.
- **Resizing Activities:** To support multi-window mode, you should enable your activities to resize. This can be done by setting the `android:resizeableActivity` attribute to "true" in your manifest file. It's also important to test your app in different multi-window configurations to ensure correct behavior and appearance.
- **Resource Qualifiers:** You can use resource qualifiers to provide different resources for different screen sizes. For example, you might have different layouts for the folded and unfolded states of a device. By appending qualifiers like "small", "normal", "large", and "xlarge" to your resource directory names, you can specify which resources should be used for which screen sizes.

Handling Screen Ratios: Ensure that your app can handle different screen aspect ratios. Android's `maxAspectRatio` and `minAspectRatio` attributes allow you to specify the maximum and minimum aspect ratios that your app supports.

In conclusion, the transition to foldable screens requires careful consideration in both design and coding aspects of your app. It might require a departure from traditional app

development practices, but by preparing your app for these changes, you can ensure a smooth, enjoyable user experience across all device types. In the next section, we'll discuss how to set up a testing environment to validate your app's behavior on foldable screens.

Setting up the environment

Setting up a testing environment for foldable screens involves both hardware and software considerations. Given the diversity of foldable devices available, emulating these environments becomes crucial. Android provides the necessary tools to support development for foldable and dual-screen devices, including the Android Emulator, which allows you to emulate foldable devices.

Hardware and Software Requirements

To start, you'll need the following:

- Experience building Android apps.
- Familiarity with Activities, Fragments, View binding, and XML-layouts1.
- Android Studio Arctic Fox or higher.

Jetpack WindowManager

The Jetpack WindowManager library aids in supporting new device form factors. It provides a common API surface for various WindowManager features on both old and new platform versions. Its key features include the `FoldingFeature` class that describes a fold in a flexible display or a hinge between two physical display panels (`1multi-windowmode`). Unlike single screen devices, foldable devices also support different postures. Postures can be used to display content in different ways.

Foldable devices can offer different spanning postures when an app is spanned (displayed) across the whole display region (using all display regions on dual-screen foldable devices). Foldable devices can also offer folded postures, like tabletop mode, so you can have a logical split between the part of the screen that's flat and the part that's tilted towards you, and tent mode, so you can visualize the content as if the device was using a stand gadget. The Jetpack WindowManager library helps application developers support new device form factors and provides a common API surface for various WindowManager features on both old and new platform versions.

Jetpack WindowManager version 1.0.0 contains the `FoldingFeature` class that describes a fold in a flexible display or a hinge between two physical display panels. Its API provides access to important information related to the device: `state()`: Provides the current posture of the device from a list of defined postures (`FLAT` and `HALF_OPENED`). `isSeparating()`: Calculates if a `FoldingFeature` should be thought of as splitting the window into multiple physical areas that can be seen by users as logically separate. `occlusionType()`: Calculates the occlusion mode to determine if a `FoldingFeature` occludes